

InspectR: PoC

Compliance automation for data-pipeline and agentic workflow ecosystems

1. What the PoC demonstrates

InspectR is a proof-of-concept architecture for evaluating selected compliance requirements against static JSON descriptions of data pipelines.

The central problem is the gap between contextual requirements and technical pipeline representations. Requirements such as approved data sources, traceability, risk checks, and human oversight must be translated into explicit evidence that a system can assess.

Core claim: InspectR demonstrates how selected requirements can be transformed into repeatable, evidence-bound, and explainable design-time checks. It does not claim to determine complete legal compliance.

2. The TK/MUNDAT use case

The TK/MUNDAT scenario is the bounded case used to demonstrate the architecture. The pipeline contains seven stages:

1. TK request intake
2. Request validation
3. Retrieval of relevant data
4. Data processing for compliance
5. Compliance checks
6. Preparation of a sharing decision
7. Decision output

The case demonstrates the method in a realistic data-sharing context. It does not represent the complete GDPR or DataPACT compliance space.

3. How the architecture works

```
JSON pipeline blueprint
↓
InspectR evaluator
↓
OPA Policy-as-Code engine
↓
Rego rules
↓
Structured result: status, reason, evidence, remedy
```

Compliance logic is kept in Rego and evaluated by OPA rather than being hard-coded into the Streamlit interface. This separates the application, policy execution, and evidence representation.

The current rules check pipeline structure, source provenance, logging declarations, the presence of a compliance-check stage, and human-oversight evidence.

4. The two JSON files

The evaluation uses two uploaded files:

- The original inLUMEN pipeline export. [inlumen-tk-uc.json](#)
- A synthetic extended version that preserves the pipeline information and adds compliance-oriented metadata and descriptions. [inlumen-tk-uc-extended.json](#)

This is a controlled comparison rather than a comparison of unrelated pipelines:

```
Same pipeline structure
Same Rego policy
Same evaluation engine
Different available compliance evidence
```

The extended file is a research artifact. It is not presented as a fully compliant production pipeline.

5. How to run the evaluation

1. Open the Evaluation page.
2. Upload the original JSON under **Blueprint 1**.
3. Click **Run InspectR on blueprint 1**.
4. Upload the extended JSON under **Blueprint 2**.
5. Click **Run InspectR on blueprint 2**.
6. Inspect the two live result columns.
7. Open a rule result to show its status, legal reference, reason, evidence, and remedy.
8. Scroll to the comparison table, which is generated only after both evaluations finish.

Important: the page does not use precomputed result values. Each uploaded file is passed to the evaluator and assessed by the current OPA/Rego policy at runtime.

6. Understanding the results

Status	Meaning	Possible workflow response
OK	The evidence required by the encoded rule was found and satisfied.	Continue, subject to other controls.
FAIL	Evidence was found but did not satisfy the rule, or it represented a negative condition.	Block, revise, or request corrective action.

HUMAN	The available evidence is insufficient for an automated conclusion.	Escalate to a human or request more evidence.
--------------	---	---

These statuses apply to the encoded rules only. **OK** does not prove complete legal or organisational compliance.

7. The deliberate human-oversight demonstration

The extended blueprint deliberately exposes a human-oversight issue. Its evidence explicitly represents:

```
human_oversight: false
```

This is different from the original file, where the evidence is missing:

```
Missing evidence → HUMAN → human review is required
Explicit negative evidence → FAIL → a known negative condition is identified
```

This behaviour is intentional. InspectR does not treat the presence of metadata as automatic compliance. It evaluates what the metadata says and surfaces a governance issue rather than hiding it.

8. What the live comparison demonstrates

- The same policy can be applied to multiple uploaded blueprints.
- Results depend on the evidence represented in each blueprint.
- Policy logic is separated from application logic.
- Results are structured and explainable.
- Missing evidence can be distinguished from explicit negative evidence.
- The same input and unchanged policy can produce repeatable results.

This supports the narrower claim that InspectR demonstrates evidence-bound compliance automation at design time.

9. Relevance to agentic workflows

```
Agent proposes or discovers a workflow
↓
Workflow becomes a pipeline blueprint
↓
InspectR evaluates the blueprint
↓
Structured result is returned
↓
Agent continues, revises, blocks, or requests human review
```

The current PoC implements the compliance-assessment and decision-support component. It does not yet implement a complete autonomous agent loop. The result semantics provide a foundation for safe

orchestration because an agent can distinguish “proceed,” “correct,” and “escalate.”

10. Limitations and conclusion

The PoC evaluates static design-time evidence. It does not observe runtime behaviour, verify that declarations are true, interpret all legal requirements, cover all GDPR or DataPACT requirements, enforce policy in a production engine, or prove scalability across heterogeneous platforms.

Conclusion: InspectR demonstrates a feasible architecture for connecting contextual requirements, pipeline evidence, Policy-as-Code rules, uncertainty-aware results, explanations, and human escalation. The TK/MUNDAT use case shows how the architecture can be demonstrated and evaluated without claiming complete legal compliance.

Suggested closing sentence: “The contribution is not OPA or Rego themselves, but their structured integration with requirement interpretation, evidence, uncertainty, explainability, and human decision support for data-pipeline compliance automation.”